

UNIVERSIDAD DE GUADALAJARACentro Universitario de Ciencias Exactas e Ingenierías

REPORTE DE PRÁCTICA

Proyecto Final — Fase 3Instrucciones Tipo J, Pipeline de 5 Buffers y Subrutinas MIPS 32 bits

Materia:	Arquitectura de Computadoras
Profesor:	Jorge Ernesto López Arce Delgado
Integrantes:	Diego Israel González Sánchez Derek Gabriel Casillas López Iván Samuel Ortiz Orellana Omar Luna Reyes
Fecha:	09 de mayo de 2026

1. INTRODUCCIÓN

1.1 Contexto de la Fase 3

Las Fases 1 y 2 del proyecto establecieron un procesador MIPS de 32 bits capaz de ejecutar instrucciones tipo R e instrucciones tipo I. La Fase 3 completa el repertorio básico del ISA MIPS implementando el formato Tipo J: las instrucciones **j** (Jump) y **jal** (Jump and Link), así como **jr** (Jump Register), que es el complemento natural de **jal** para el retorno de subrutinas. Adicionalmente se extiende la infraestructura de pipeline añadiendo un quinto buffer (WB/PC), siguiendo el capítulo 4.6 de Patterson & Hennessy, donde todos los registros de pipeline se disparan con el mismo reloj que el PC.

1.2 Formato de instrucción Tipo J

Las instrucciones MIPS de tipo J tienen un formato de 32 bits dividido en dos campos: opcode de 6 bits y dirección de 26 bits. La dirección efectiva de salto se construye concatenando los 4 bits más significativos del PC+4 con los 26 bits del campo desplazados 2 posiciones a la izquierda:

```
jump_addr = { PC+4[31:28], instruction[25:0], 2'b00 }
```

Campo	Bits	j (opcode)	jal (opcode)	jr (funct)	Ancho
opcode	31–26	000010	000011	000000	6 bits
target/rs	25–0 / 25–21	Dirección de 26 bits	Dirección de 26 bits	rs (registro con dir.)	26/5 bits

1.3 Instrucciones J, JAL y JR

- **j label** — Salta incondicionalmente a la dirección calculada. Opcode 000010. No modifica ningún registro.
- **jal label** — Igual que **j**, pero guarda PC+4 en \$ra (\$31) automáticamente. Opcode 000011. Permite llamadas a subrutinas.
- **jr \$ra** — Tipo R (funct=001000). El PC toma el valor del registro rs. Usualmente **jr \$ra** para retornar de una subrutina invocada con **jal**.

2. INVESTIGACIÓN: PIPELINE MIPS — CAPÍTULO 4.6 P&H

2.1 Concepto y motivación

Un procesador single-cycle ejecuta cada instrucción completamente en un ciclo, desperdiciando recursos: mientras la ALU opera, el banco de registros y la memoria están inactivos. El tiempo de ciclo debe ser igual a la instrucción más lenta (lw), penalizando todas las demás. El pipeline divide la ejecución en etapas independientes y permite que varias instrucciones se ejecuten simultáneamente, cada una en una etapa diferente, aumentando el throughput en un factor aproximado al número de etapas.

2.2 Las cinco etapas del pipeline MIPS

- **IF — Instruction Fetch:** El PC entrega su dirección a la memoria de instrucciones. Se lee la instrucción de 32 bits y se calcula PC+4.
- **ID — Instruction Decode / Register Read:** Se decodifica el opcode, la unidad de control genera señales, y el banco de registros lee rs y rt simultáneamente. Se extiende en signo el inmediato de 16 bits.
- **EX — Execute / Address Calculation:** La ALU realiza la operación o calcula la dirección de memoria. Se calculan también las direcciones objetivo de branch y jump.
- **MEM — Memory Access:** Acceso a la memoria de datos para lw (lectura) o sw (escritura). La señal Branch se combina con zero para decidir saltos condicionales.
- **WB — Write Back:** El resultado se escribe en el registro destino. Para JAL, se escribe PC+4 en \$ra.

2.3 Registros de pipeline (buffers)

Para separar físicamente las etapas se insertan registros síncronos entre cada par de etapas adyacentes. Estos buffers capturan en su flanco positivo de reloj todas las señales (datos y control) producidas por la etapa anterior y las entregan a la etapa siguiente en el ciclo siguiente. El punto crítico de P&H capítulo 4.6: todos los registros de pipeline, incluyendo el PC, deben dispararse con el mismo flanco del mismo reloj.

2.4 Los cinco buffers de Fase 3

- **Buffer 1 — IF/ID:** Almacena PC+4 e instrucción de 32 bits al final de IF.
- **Buffer 2 — ID/EX:** El más amplio. Propaga señales de control, datos del banco de registros, inmediato extendido, campos de instrucción (rs,rt,rd,funct,opcode,shamt) y campo de 26 bits para J/JAL.
- **Buffer 3 — EX/MEM:** Captura resultado ALU, flag zero, direcciones de branch y jump, dato para sw, registro destino y PC+4 para JAL.
- **Buffer 4 — MEM/WB:** Propaga dato leído de memoria, resultado ALU, señales de write-back y registro destino.
- **Buffer 5 — WB/PC (nuevo Fase 3):** Retarda la señal de salto J/Branch un ciclo adicional para alinearla con el flanco del PC. Sin este buffer la decisión de salto llegaría al Mux del PC con un ciclo de desfase.

2.5 Hazards en pipeline

- **Hazards de datos:** Una instrucción necesita el resultado de una anterior aún no completada. Solución: forwarding (reenvío directo desde EX/MEM a la ALU). No implementado en esta fase; los programas de prueba se diseñan para evitarlos.
- **Hazards de control:** Las instrucciones de salto hacen que el procesador busque instrucciones incorrectas. El buffer WB/PC es la primera aproximación; en un pipeline completo se usaría flush o predicción de salto.

- **Hazards estructurales:** MIPS los elimina por diseño con memorias separadas de instrucciones y datos (Harvard modificada).

3. OBJETIVO

3.1 Objetivo General

Extender el datapath MIPS de 32 bits de la Fase 2 para soportar instrucciones tipo J (j y jal) y jr, implementando el quinto buffer de pipeline (WB/PC) conforme al capítulo 4.6 de P&H, y validando el diseño mediante la ejecución del algoritmo de test de primalidad que usa JAL para llamadas a subrutina y JR para el retorno.

3.2 Objetivos Particulares

- Implementar señales Jump y JAL en Control.v para opcodes 000010 (j) y 000011 (jal).
- Diseñar Mux3.v para selección del PC entre PC+4, branch target y jump address.
- Actualizar los cuatro buffers existentes para propagar Jump, JAL, jump_target y pc_plus4.
- Implementar el quinto buffer WBPC_Buffer.v que alinea la señal de salto con el flanco del PC.
- Implementar lógica JR en MIPS_Top: detectar funct=001000 y redirigir el PC al contenido de \$rs.
- Implementar en WB la escritura de PC+4 en \$ra cuando JAL=1, con MUX de dato y MUX de registro destino.
- Codificar el test de primalidad en ensamblador MIPS usando j, jal y jr, y generar TestF3_MemInst.mem.

4. DESARROLLO

4.1 Unidad de Control — Ecuaciones Lógicas para Instrucciones Tipo J

La Unidad de Control es un circuito combinacional puro. Para las instrucciones tipo J las ecuaciones lógicas en función del opcode son:

```
Jump      = (opcode == 000010) | (opcode == 000011) // j o jal
JAL       = (opcode == 000011) // solo jal
RegWrite  = (op==000000) | (op==001000) | (op==001010) | (op==001100)
           | (op==001101) | (op==100011) | (op==000011) // incluye jal
Branch    = (opcode == 000100) // beq
```

Tabla completa de señales de control por instrucción:

Opcode	Instruc.	RegDst	ALUSrc	MemtoReg	RegWr	Branch	Jump	JAL	ALUOp
000000	Tipo R	1	0	0	1	0	0	0	10
001000	addi	0	1	0	1	0	0	0	00
001010	slti	0	1	0	1	0	0	0	11
001100	andi	0	1	0	1	0	0	0	01
001101	ori	0	1	0	1	0	0	0	01
100011	lw	0	1	1	1	0	0	0	00
101011	sw	X	1	X	0	0	0	0	00
000100	beq	X	0	X	0	1	0	0	01
000010	j	X	X	X	0	0	1	0	XX
000011	jal	X	X	X	1	0	1	1	XX
000000*	jr	1	0	0	0	0	1	0	XX

4.2 ALU — Operaciones Soportadas y Codificación

La ALU es un bloque combinacional que recibe operandos a y b de 32 bits, el campo shamt de 5 bits y alu_control de 4 bits. Para J/JAL la ALU no participa en el cálculo de la dirección de salto.

alu_control	Operación	Instrucciones	Expresión
0000	AND	and, andi	result = a & b
0001	OR	or, ori	result = a b
0010	ADD	add, addi, lw, sw	result = a + b
0011	XOR	xor, xori	result = a ^ b
0110	SUB	sub, beq	result = a - b (zero=1 si a==b)
0111	SLT	slt, slti	result = (a < b) ? 1 : 0
1000	SLL	sll	result = b << shamt
1001	SRL	srl	result = b >> shamt (lógico)
1100	NOR	nor	result = ~(a b)

4.3 Memoria de Instrucciones — Organización y Mapeo

ROM de 256 palabras de 32 bits. Lectura combinacional (asíncrona): la instrucción está disponible en el mismo ciclo sin esperar un flanco de reloj.

```
assign instruction = mem[address[9:2]]; // bits [9:2] del PC
```

Índice	Dir. byte (hex)	PC (dec)	Instrucción (Fase 3 — primalidad)
0	0x00000000	0	lw \$t0,16(\$zero) — carga N desde MEM[4]
5	0x00000014	20	[outer_chk] slt \$t3,\$t6,\$t1
7	0x0000001C	28	jal mod_sub — PC+4=32 se guarda en \$ra
10	0x00000028	40	j outer_chk — salto incondicional al lazo
13	0x00000034	52	[halt] j halt — bucle infinito (FIN)
14	0x00000038	56	[mod_sub] add \$t2,\$t0,\$zero — entrada subrutina
19	0x0000004C	76	[mod_ret] jr \$ra — retorno al llamador

4.4 Banco de Registros — Estructura y Manejo de \$ra en JAL

32 registros de 32 bits. \$zero (reg 0) protegido de escritura. \$ra (reg 31) es físicamente igual a cualquier otro registro; su rol especial en JAL lo define la etapa WB con dos MUX adicionales:

```
wb_write_data = wb_JAL ? wb_pc_plus4 : wb_alu_or_mem; // dato
wb_write_reg_f = wb_JAL ? 5'd31 : wb_write_reg; // destino
```

Puerto	Tipo	Descripción
read_data1 (rs)	Combinacional	registers[rs] disponible en el mismo ciclo
read_data2 (rt)	Combinacional	registers[rt] disponible en el mismo ciclo
write (rd / \$ra)	Síncrono ↑CLK	Si RegWrite=1 y destino≠0. Para JAL: destino=31, dato=PC+4

4.5 PC y Lógica de Salto — Circuito Detallado y Ecuaciones

El PC es el único registro síncrono de la etapa IF. El Mux3 selecciona entre tres fuentes usando señales retardadas por el quinto buffer (WB/PC):

```
// Selección del PC (Mux3):
next_pc = pc_jump_taken ? jump_addr : // J, JAL, JR
         pc_branch_taken ? branch_target : // BEQ tomado
         pc_plus4; // secuencial

// Dirección J/JAL (etapa EX):
jump_addr = { ex_pc_plus4[31:28], ex_jump_target[25:0], 2'b00 }

// Dirección JR (etapa EX, funct=001000):
jump_addr_jr = ex_read_data1 // contenido de $ra

// Dirección BEQ (etapa EX):
branch_target = ex_pc_plus4 + (SignExt(imm16) << 2)

// Quinto buffer alinea decision con flanco del PC:
pc_sel = f_jump ? 2'b10 : f_branch ? 2'b01 : 2'b00;
```

Señal	Origen en pipeline	Destino
pc_plus4	Adder etapa IF	Buffer IF/ID → ... → MEM/WB → \$ra (JAL)
jump_addr	Concatenación etapa EX	Buffer EX/MEM → WB/PC → Mux3

branch_target	Adder branch etapa EX	Buffer EX/MEM → WB/PC → Mux3
pc_sel[1:0]	Lógica combinacional MIPS_Top	Entrada de selección Mux3

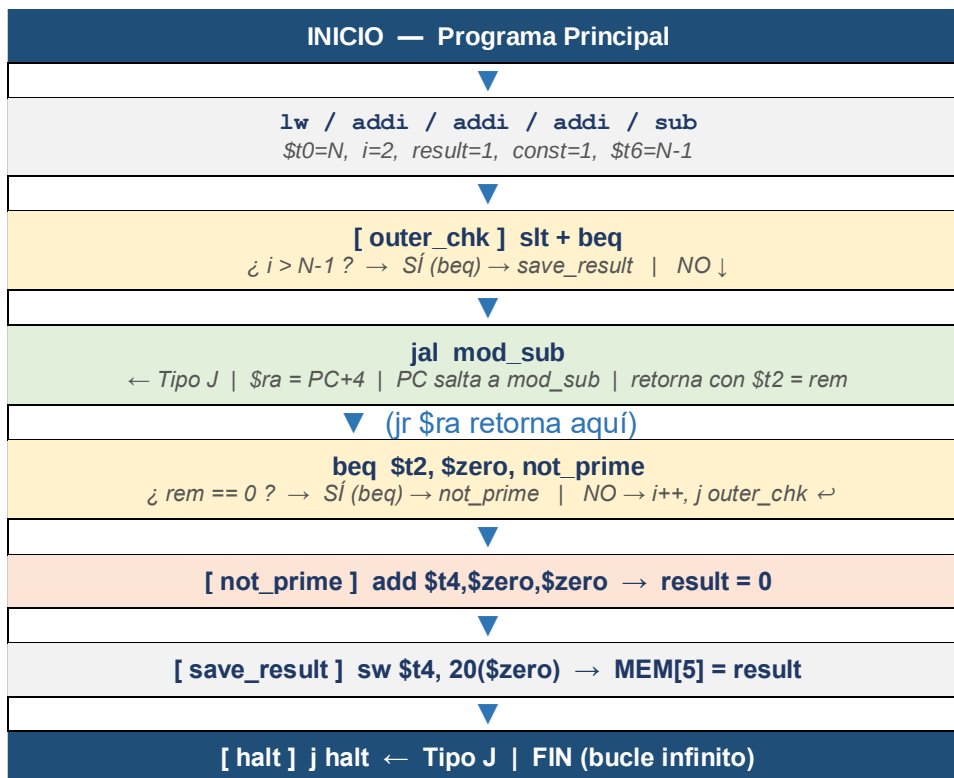
5. ALGORITMO: TEST DE PRIMALIDAD CON SUBROUTINA

5.1 Fundamentos teóricos

El test de primalidad por divisiones de prueba (trial division) determina si N es primo verificando si existe algún entero d con $2 \leq d \leq N-1$ tal que $N \bmod d = 0$. Sin la instrucción `div`, el módulo se calcula por restas sucesivas: partir de $\text{rem}=N$ y restar d repetidamente mientras $\text{rem} \geq d$. Si $\text{rem}=0$ al terminar, d divide a N (N es compuesto).

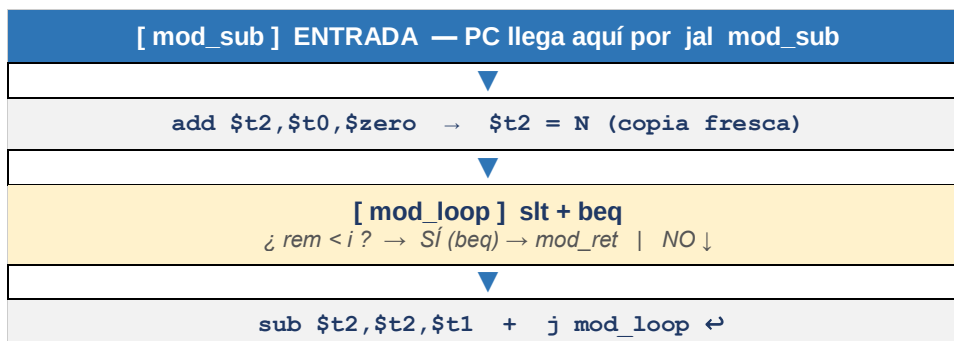
La extracción del cálculo del módulo como subrutina (`mod_sub`) demuestra el mecanismo completo JAL+JR: el llamador ejecuta '`jal mod_sub`' guardando automáticamente $\text{PC}+4$ en $\$ra$, y la subrutina retorna con '`jr $ra`'.

5.2 Diagrama de flujo — Programa Principal



5.3 Diagrama de flujo — Subrutina mod_sub

Invocada con JAL, retorna con JR \$ra. Calcula $\$t2 = \$t0 \bmod \$t1$ por restas sucesivas.



$rem = rem - i$ (continúa restando)
▼
[mod_ret] jr \$ra ← Tipo R (funct=001000) PC = \$ra RETORNO

5.4 Trazas de ejecución

N=7 (primo): $i=2 \rightarrow rem=1$, $i=3 \rightarrow rem=1$, $i=4 \rightarrow rem=3$, $i=5 \rightarrow rem=2$, $i=6 \rightarrow rem=1$. $i=7 > N-1=6 \rightarrow \text{save_result}$. MEM[5]=1. ✓

N=4 (no primo): $i=2 \rightarrow rem=4-2-2=0 \rightarrow \text{not_prime}$. MEM[5]=0. ✓

6. RESULTADOS Y VALIDACIÓN

6.1 Instrucciones J/JAL/JR en TestF3_MemInst.mem

#	Instrucción	Tipo	Campo crítico	Binario (32 bits)
7	jal mod_sub	J (000011)	target=14	00001100000000000000000000001110
10	j outer_chk	J (000010)	target=5	0000100000000000000000000000101
13	j halt	J (000010)	target=13	00001000000000000000000000001101
18	j mod_loop	J (000010)	target=15	00001000000000000000000000001111
19	jr \$ra	R (001000)	rs=\$ra(31)	00000011111000000000000000001000

6.2 Resultados esperados en simulación

Señal/Registro	N=7 (primo)	N=4 (no primo)	Instrucción
MEM[5] resultado	1	0	sw \$t4,20(\$zero)
\$ra (reg 31)	32 = 0x00000020	32 = 0x00000020	jal idx 7 → PC+4=32
PC tras jr \$ra	0x20 = idx 8	0x20 = idx 8	jr \$ra
PC en halt	0x34 = idx 13	0x34 = idx 13	j halt

6. DECODIFICADOR PYTHON Y FLUJO ASM → MEM → VERILOG

6.1 Flujo completo: de ensamblador a simulación

El flujo de trabajo del proyecto sigue tres pasos claramente definidos. Primero, el programador escribe el algoritmo en lenguaje ensamblador MIPS (.asm) con etiquetas, comentarios y mnemónicos legibles. Segundo, el decodificador Python (assembler.py) traduce cada instrucción a su representación binaria de 32 bits y genera un archivo .mem. Tercero, el simulador Verilog (ModelSim) carga ese archivo en la memoria de instrucciones usando la directiva \$readmemb y ejecuta el programa.

	Paso	Herramienta	Entrada	Salida
1	Escritura del algoritmo	Editor de texto	— (diseño)	primality_test_fase3.asm
2	Ensamblado a binario	assembler.py (Python)	.asm	TestF3_MemInst.mem
3	Carga en simulador	\$readmemb (Verilog)	.mem	mem[] en InstructionMemory
4	Simulación	ModelSim	Proyecto Verilog	Formas de onda / resultado

6.2 Formato \$readmemb — modo binario en Verilog

La directiva \$readmemb (read memory binary) de Verilog lee un archivo de texto donde cada línea contiene exactamente una palabra en formato binario. Cada '0' y '1' en el archivo corresponde directamente a un bit del arreglo de memoria. Esta es la razón por la que el decodificador Python genera cadenas de 32 caracteres '0'/'1' en lugar de hexadecimal.

```
// InstructionMemory.v — carga con $readmemb:
initial begin
    $readmemb("TestF3_MemInst.mem", mem);
end

// Regla: cada línea del .mem = una posición del arreglo mem[]
// Línea 0 → mem[0] → instrucción en PC=0x00
// Línea 1 → mem[1] → instrucción en PC=0x04
// Línea N → mem[N] → instrucción en PC=N*4
```

El archivo .mem también puede contener comentarios con // — Verilog los ignora al leer. Esto permite documentar cada instrucción directamente en el archivo de memoria, como se hace en TestF3_MemInst.mem:

```
// TestF3_MemInst.mem — ejemplo de formato
// idx 0: lw $t0, 16($zero)
1000110000000100000000000000010000
// idx 1: addi $t1, $zero, 2
00100000000010010000000000000010
// idx 7: jal mod_sub (opcode=000011, target=14)
00001100000000000000000000001110
```

6.3 Cómo el decodificador Python genera el formato \$readmemb

El assembler.py realiza dos pasadas (two-pass assembler). En la primera pasada recorre todas las líneas del .asm recolectando las etiquetas y asignándoles el índice de instrucción correspondiente. En la segunda pasada codifica cada instrucción a binario de 32 bits resolviendo los saltos con las etiquetas ya conocidas.

```
# Primera pasada - recolectar etiquetas
def first_pass(lines):
    pc = 0 # índice de instrucción
    for line in lines:
        if 'label:' in line:
            labels[label] = pc # guarda índice
        if tiene_instruccion:
            pc += 1

# Segunda pasada - codificar instrucciones
def encode(mnemonic, ops, pc_idx, labels):
    if mnemonic == 'j':
        target_field = labels[label] # índice de instrucción
        return bin(opcode,6) + bin(target_field,26)
    if mnemonic == 'jal':
        # igual que j pero opcode=000011
        ...
    # Cada instrucción retorna string de 32 '0'/'1'
```

La función `to_bin()` convierte enteros a strings binarios de longitud fija, manejando correctamente los números negativos (complemento a dos) para offsets de branch negativos:

```
def to_bin(value, bits):
    if value < 0:
        value = value & ((1 << bits) - 1) # complemento a 2
    return format(value, f'0{bits}b')

# Ejemplo: offset = -4 → to_bin(-4, 16)
# -4 & 0xFFFF = 65532 = 1111111111111100 ✓
```

6.4 Codificación de instrucciones J y JAL en el decodificador

Las instrucciones tipo J son las más simples de codificar: solo tienen opcode de 6 bits y el campo target de 26 bits. El campo target es el índice de instrucción (no la dirección byte) de la etiqueta destino, ya que Verilog accede al arreglo por índice y no por dirección byte:

```
# Tipo J - j y jal
if mnemonic in ('j', 'jal'):
    opcode = 0b000010 if mnemonic=='j' else 0b000011
    target_raw = ops.strip()
    if target_raw in labels:
        target_idx = labels[target_raw] # índice instrucción
        target_addr = target_idx * 4    # dirección byte
        target_field = target_addr >> 2 # campo de 26 bits
    return to_bin(opcode,6) + to_bin(target_field,26)
```

El campo de 26 bits que se codifica en la instrucción es la dirección byte dividida entre 4 (equivale al índice de instrucción). Cuando el procesador ejecuta el salto, reconstruye la dirección completa de 32 bits así:

```
// Hardware Verilog - reconstrucción de dirección de salto:
jump_addr = { ex_pc_plus4[31:28], // 4 bits MSB del PC+4
              ex_jump_target[25:0], // 26 bits del campo
              2'b00 };             // alinear a 4 bytes

// Ejemplo: jal mod_sub (idx=14)
// target_field = 14 → binario 26 bits = 00...01110
// jump_addr = {PC+4[31:28], 00000000000000000001110, 00}
// jump_addr = 0x00000038 (dirección byte 56 = idx 14 × 4) ✓
```

6.5 Uso del decodificador desde línea de comandos

El archivo `assembler.py` acepta el `.asm` como entrada y genera el `.mem` compatible con `$readmemb`. Tiene tres modos de salida:

```
# Generar .mem desde .asm (modo normal):
python3 assembler.py primality_test_fase3.asm -o TestF3_MemInst.mem

# Ver binario con decodificación de vuelta para verificar:
python3 assembler.py primality_test_fase3.asm --decode

# Ver también en hexadecimal:
python3 assembler.py primality_test_fase3.asm --hex
```

Ejemplo de salida con --decode, mostrando las instrucciones J/JAL/JR verificadas:

```
// Generado por assembler.py - MIPS Fase 3 CUCEI
// Total instrucciones: 20
0000110000000000000000000000001110 // [ 7] jal 56
000010000000000000000000000000101 // [10] j 20
000010000000000000000000000000101 // [13] j 52
000010000000000000000000000000111 // [18] j 60
0000001111100000000000000000001000 // [19] jr $ra
```

8. TABLA DE SEÑALES DE CONTROL — ISA COMPLETO FASE 3

Instruc.	RegDst	ALUSrc	MemtoReg	RegWr	MemRd	MemWr	Branch	Jump	JAL
Tipo R	1	0	0	1	0	0	0	0	0
addi	0	1	0	1	0	0	0	0	0
slti	0	1	0	1	0	0	0	0	0
andi	0	1	0	1	0	0	0	0	0
ori	0	1	0	1	0	0	0	0	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	0
j	X	X	X	0	0	0	0	1	0
jal	X	X	X	1	0	0	0	1	1
jr	1	0	0	0	0	0	0	1	0

9. CONCLUSIONES

Diego Israel González Sánchez

La implementación de las instrucciones tipo J completó el ciclo de diseño del ISA MIPS básico iniciado en la Fase 1. El aspecto más relevante fue comprender que el cálculo de la dirección de salto J utiliza los 4 bits más significativos del PC+4 — no del PC actual — lo que implica que el ensamblador debe calcular el campo target en función de la instrucción siguiente. Esta sutileza es fundamental para codificar manualmente instrucciones J sin errores. El quinto buffer demostró que el pipeline no es solo una cuestión de latencia de datos sino también de sincronización temporal de señales de control.

Derek Gabriel Casillas López

La implementación del mecanismo JAL+JR fue la contribución más representativa de esta fase. JAL es el puente entre el hardware (el procesador guarda automáticamente la dirección de retorno) y el software (el programador puede escribir funciones reutilizables). Entender que \$ra no es un registro especial en el hardware — es simplemente el registro 31 — pero sí lo es por convención del compilador, ilustra perfectamente la interfaz hardware/software que es el tema central de Patterson y Hennessy.

Iván Samuel Ortiz Orellana

La investigación del capítulo 4.6 de Computer Organization and Design reveló que el pipeline de MIPS no es una optimización añadida sobre el diseño single-cycle, sino un cambio arquitectónico fundamental. El hecho de que todos los buffers deban dispararse con el mismo flanco del mismo reloj que el PC no es una restricción arbitraria sino una consecuencia directa del modelo de computación síncrona: en cada ciclo, exactamente una 'fotografía' del estado del procesador avanza a través del pipeline.

Omar Luna Reyes

El diseño del algoritmo de test de primalidad con subrutina jal/jr demostró que, incluso con un ISA mínimo, es posible implementar programas estructurados y reutilizables. La decisión de extraer el cálculo del módulo como subrutina sirve para demostrar JAL+JR y también hace el código más legible. Esto refleja los principios de programación modular que los compiladores C aplican sistemáticamente: cada función se compila con un prólogo y un epílogo (jr \$ra).

Conclusión General

La Fase 3 cierra el ciclo del Proyecto Final con un procesador MIPS de 32 bits que soporta el conjunto completo de instrucciones básicas: tipo R, tipo I y tipo J. La arquitectura de 5 buffers de pipeline prepara el camino para una Fase 4 donde se implementaría el pipeline completo con detección de hazards de datos (forwarding) y de control (flush en saltos), transformando el procesador en un sistema de alto rendimiento real.

10. REFERENCIAS

- [1] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5.^a ed. Morgan Kaufmann, 2014. Caps. 4.3, 4.4, 4.6.
- [2] MIPS Technologies, *MIPS32 Architecture for Programmers Vol. II: The MIPS32 Instruction Set*. MIPS Technologies, 2001.
- [3] S. Brown y Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3.^a ed. McGraw-Hill, 2014.
- [4] D. M. Harris y S. L. Harris, *Digital Design and Computer Architecture*, 2.^a ed. Morgan Kaufmann, 2012.
- [5] Intel/Altera, *ModelSim-Altera Software Simulation User Guide*. Altera Corporation, 2016.
- [6] CUCEI, Universidad de Guadalajara, *Reporte de Práctica Proyecto Final — Fases 1 y 2*. Guadalajara, México, 2026.